FINAL PROJECT

Team members / NetIDs	Section# / instructor
_____	
_____	_____

Project at a glance

In this team project you will reuse skills from CSE 1106 labs.

When	Milestone	Evidence
Week 1 •	Choose product; define responsibilities; create repository; draft requirements and diagram.	GitHub link + 1-page plan.
	Build working C prototype; compile and run on Pi; save data; start tests; commit.	Live prototype check + initial commit.
Week 2 •	Complete the test plan; fix bugs; prepare LED circuit and documentation.	Five test cases + issue/change log.
	Connect software to indicator; regression-test; finalize documentation; demo.	Final repository + live instructor verification.

1. LAB SESSION A — Prototype your product (3 hours)
    a. Before creating code, create a new GitHub repository with a README (following Lab 2).
    b. Follow the following steps to interface your RPi.
    c. Choose 1 out of the 3 projects listed.

2. LAB SESSION B —(3 hours)
    a. Plan and code your chosen project.
    b. List your test cases and run those tests for the instructor available.

Part A: GitHub Project Set Up

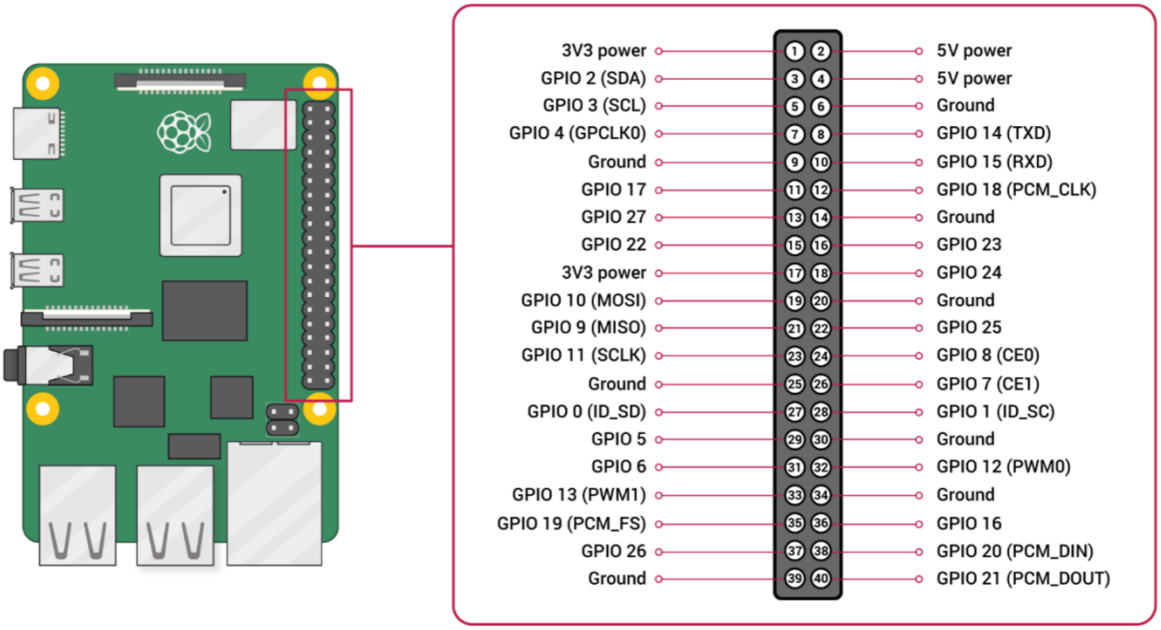
1. Before creating code, create a new GitHub repository with a README (following Lab 2). Clone the repository.

Recommended repository structure

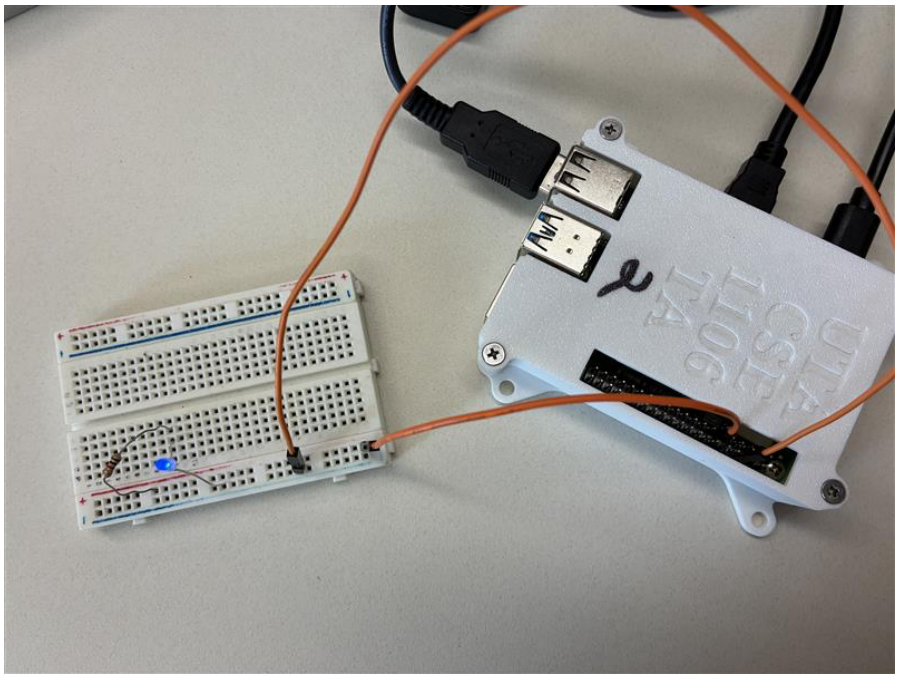
```
cse1106-final/
├── src/
│   └── main.c
├── docs/
│   ├── requirements.md
│   ├── design.md
│   └── change_log.md
├── tests/
│   └── test_plan.md
├── data/
│   └── temperature_log.csv
├── led_indicator.py      # supplied helper
├── status.txt            # NORMAL or WARNING
└── README.md
```

Part B: RPi Interface Template

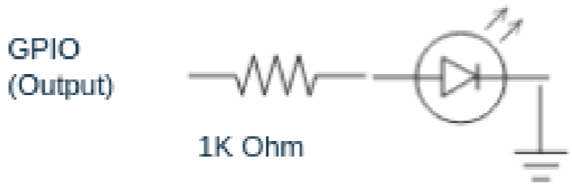
1. On the RPi 4b, you will find a 40-pinheader when the top lid is removed as shown below:



2. Connect the appropriate wires to the RPi. Following the image below:



For our template experiment, let's connect a red LED to a pin on the RPi. Using a black wire, connect one of the GND pins on the RPi to the black wires on the white breadboard so that ground (0V) is connected. Then using the circuit below with R1 = 1000 ohms, connect Vin using any color wire except red or black to any of the GPIOx pins above (except for GPIO2, GPIO23, GPIO214, GPIO215, GPIO219 or GPIO223).



3. Switch the computer screen to access the RPi through the monitor

4. Once username and password are created you will then open the C GUI to write up your C program for the RPi

```
#include <stdio.h>
#include <unistd.h>
#include <pigpio.h>
```

```
#define PIN 17
int main(int argc, char* argv[])
{
 gpioInitialise();
 gpioSetMode(PIN,PI_OUTPUT);
 gpioWrite(PIN,1);// LED ON
 getchar(); // waits for Enter key
 gpioWrite(PIN,0);// LED OFF
 getchar(); // waits for Enter key
 gpioTerminate();
 return 0;
}
```

***Note*:** the PIN # is the number that follows the GPIO value so if you have it connected to GPIO 17 (the physical pin is 11) but the number utilized in the code is the GPIO # (which in this specific case is 17).

5. Save the above program as **led.c**, compiled with "gcc -o led led.c -lpigpio",

6. Execute with "**sudo ./led**" will turn the LED connected top GPIO 17 OFF and wait until the ENTER key is pressed. It will then turn the LED OFF and wait to exit until the ENTER key is pressed again.

Part C: Choose Project

Option 1 — Smart Classroom Comfort Monitor

Project Description

Your team has been hired to develop a simple classroom monitoring system for a university facilities department.

The department wants to monitor classroom temperatures and identify when conditions fall outside an acceptable range.

Your task is to develop a Raspberry Pi–based application that processes temperature readings, determines whether the classroom environment is acceptable, and provides a visible warning when a temperature is outside the specified range.

Expected Product Functionality

- Allow the user to enter classroom temperature readings in Fahrenheit.
- Determine whether a temperature is within the acceptable range of **68°F–76°F**, inclusive.

- Display an appropriate NORMAL or WARNING status.
- Calculate the average of the recorded temperatures.
- Save temperature readings and their statuses to a file.
- Use an LED connected to the Raspberry Pi to indicate a warning condition.
- Allow the acceptable temperature range to be changed according to a new client requirement.

Example:

Temperature	System Status	Warning LED
72°F	NORMAL	OFF
79°F	WARNING	ON
65°F	WARNING	ON

Option 2 — Smart Study Room Occupancy Tracker

Project Description

A university library wants a system that helps staff monitor the number of students occupying a study room.

Each study room has a maximum capacity. The library needs a way to track students entering and leaving and to identify when the room has reached its occupancy limit.

Your team will develop a Raspberry Pi–based occupancy tracking system that maintains the current occupancy count and provides a visual indication when the room is full.

Expected Product Functionality

- Allow the user to record when a person enters or exits a study room.
- Maintain an accurate count of current occupants.
- Prevent occupancy from becoming negative.
- Prevent the count from exceeding the maximum room capacity.
- Display the current occupancy and room status.
- Save occupancy events to a file.
- Use an LED connected to the Raspberry Pi to indicate when the room is full.
- Support a change to the maximum room capacity.

Example:

Assume the room has a maximum capacity of 5 students.

Current Occupancy	System Status	Full Indicator LED
2 / 5	SPACE AVAILABLE	OFF
4 / 5	SPACE AVAILABLE	OFF
5 / 5	ROOM FULL	ON

The program should not permit a sixth person to enter unless someone has exited.

Option 3 — Raspberry Pi Equipment Health Monitor

Project Description

A university IT department maintains Raspberry Pi devices used in classrooms and teaching laboratories.

The department wants a simple diagnostic application that can identify when a device is running low on available resources.

Your team will develop a monitoring system that examines resource utilization and warns users when a resource exceeds an acceptable threshold.

Expected Product Functionality

- Obtain or accept information about the Raspberry Pi's memory and storage usage.
- Display the current resource utilization.
- Calculate or interpret utilization percentages.
- Compare utilization against an established warning threshold.
- Display a HEALTHY or WARNING status.
- Save diagnostic results to a file.
- Use an LED connected to the Raspberry Pi to indicate a warning condition.
- Allow the warning threshold to be changed according to a new client requirement.

Example:

Assume the warning threshold is 80%.

Memory Usage	Storage Usage	System Status	Warning LED
45%	60%	HEALTHY	OFF

85%	50%	WARNING	ON
70%	92%	WARNING	ON

A warning should occur when at least one monitored resource exceeds the threshold.

Deliverable 1 — GitHub Repository

Provide a link to your team's final GitHub repository containing the source code, README, and project documentation.

Deliverable 2 — Requirements and System Design

Submit a brief document describing your product's functional requirements and its basic architecture, including how the software and hardware components interact.

Deliverable 3 — Working Program & Hardware Integration

Submit the C source files needed to compile and run your application on the Raspberry Pi.

Demonstrate the LED indicator and explain how the hardware reflects the program's output.

Deliverable 5 — Software Testing Report

Provide at least five test cases with test inputs, expected outputs, actual outputs, and pass/fail results.

Include evidence that you tested your software after implementing changes.

Deliverable 6 — Final Demonstration

Each team will deliver a **3–5 minute live demonstration** during the final scheduled laboratory session.

The demonstration should explain the product, show its main functionality, demonstrate its status indicator, and summarize testing results.

Every team member must participate and be able to explain their contribution.

Deliverable 7 — Individual Reflection

Each student must submit a short individual reflection addressing:

1. What was your primary contribution to the project?
2. What was the most challenging technical problem your team encountered?
3. How did your team use testing or debugging to solve a problem?
4. Which skills from CSE 1106 were most useful?
5. What would you improve if you had more time?